

A Parallel Algorithm for the Single Step Searching Problem

Richard Chia-Tung Lee, Jeen-Shang Lin, Fang Rong Hsu

Proceedings of the International Symposium on Parallel Architectures,
Algorithms and Networks (ISPAN), December 1994, Kanazawa, Japan

Presenter: Lin Yu-Chen

Date: Sept, 20, 2022

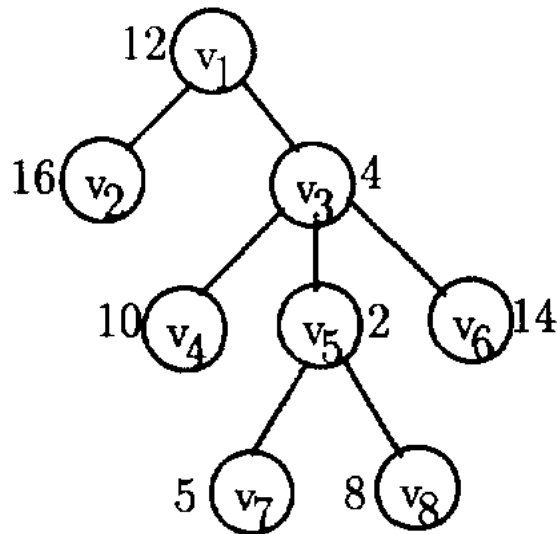
Abstract

In this paper, we shall first introduce the single step searching problem which is defined as follows: We are given a graph where each vertex is associated with a weight. Assume that every edge of graph is of equal length. A fugitive can and may be hidden in any edge. We are asked to assign searchers to vertices to search the entire graph in one step such that no fugitive can escape. The cost of a searching plan is related to the weights of the vertices in which the searchers are initially located. Our goal is to minimize the cost of the searching plan.

A parallel algorithm based upon The EREW model is proposed to solve this problem. This algorithm applies the tree contraction technique. The critical point is that we have to transform a general tree to a binary tree including pseudo nodes in order to apply this tree contraction technique. A new algorithm is devised to solve the problem on the transformed binary tree. It can be proved that this new algorithm is correct as it produces a correct solution for the original tree. Our algorithm has an optimal speed up.

Basic concept of the problem

- Given an undirected graph $G(V,E)$ with n edges and m vertices and each vertex associated with a weight $wt(v)$.
- A fugitive who can move **in any speed** is assumed to be hidden in some edge of G .
- For each edge of G , an edge searcher is assigned to search him.
- The cost of clearing an edge (u, v) is defined as $wt(u)$ if the edge searcher starts from **u** .



How to compute the cost?

We first define some notations. Let v_i be a vertex of a tree.

Case 1: v_i is a leaf node. Then $T(v_i)$ denotes v_i and its parent node.

Case 2: v_i is an internal node, but not the root node. Then $T(v_i)$ denotes the tree involving v_i , the parent node v_j and all of the descendent nodes of v_i .

Case 3: v_i is the root of a tree T , then $T(v_i)$ denotes T .

Furthermore, let v_j be the parent node of v_i . Then $C(T(v_i), v_i, v_j)(C(T(v_i), v_j, v_i))$ denotes the cost of an optimal single step searching plan where the searching direction of (v_j, v_i) is from v_i to v_j (v_j to v_i). Let r be the root of a tree. Let $C(T(r), r)(C(T(r), \bar{r}))$ be the cost of an optimal single step searching plan for T with (without) extra searcher stationed at r . The cost of an optimal single step searching plan for T is $C(T(r))$.

Rule 1: Let r be the root of a tree. $C(T(r)) = \min\{C(T(r), r), C(T(r), \bar{r})\}.$

Rule 2: Let r be the root of a tree where $v_1, v_2, \dots, v_m$ are descendants of r .

$$C(T(r), \bar{r}) = \min\left\{\sum_{i=1}^m C(T(v_i), r, v_i), \sum_{i=1}^m C(T(v_i), v_i, r)\right\}$$
$$C(T(r), r) = wt(r) + \sum_{i=1}^m \min\{C(T(v_i), r, v_i), C(T(v_i), v_i, r)\}$$

Rule 4: If u is a leaf node and w is its parent node, then

$$C(T(u), w, u) = wt(w) \text{ and } C(T(u), u, w) = wt(u)$$

Rule 3: Let u be an internal node, w be its parent node and $v_1, v_2, \dots, v_m$ be its descendant nodes.

$$C(T(u), w, u, \bar{u}) = wt(w) + \sum_{i=1}^m C(T(v_i), v_i, u)$$

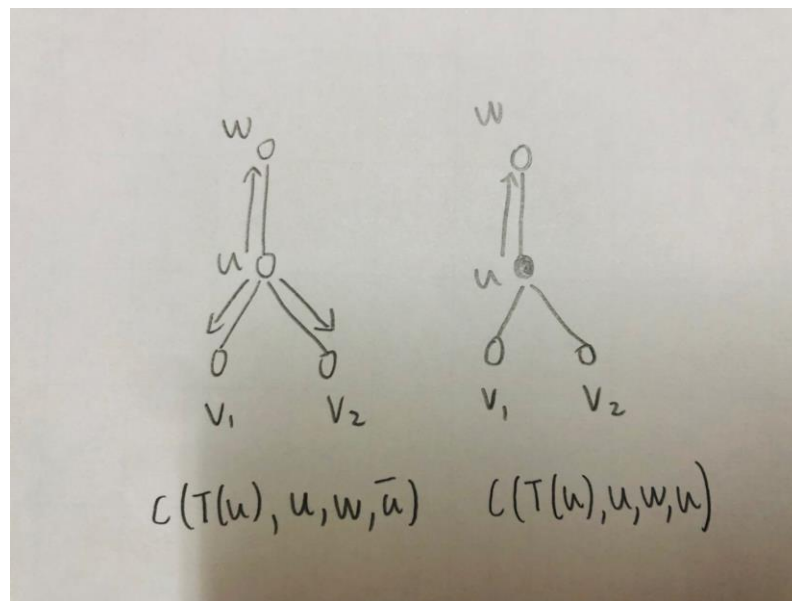
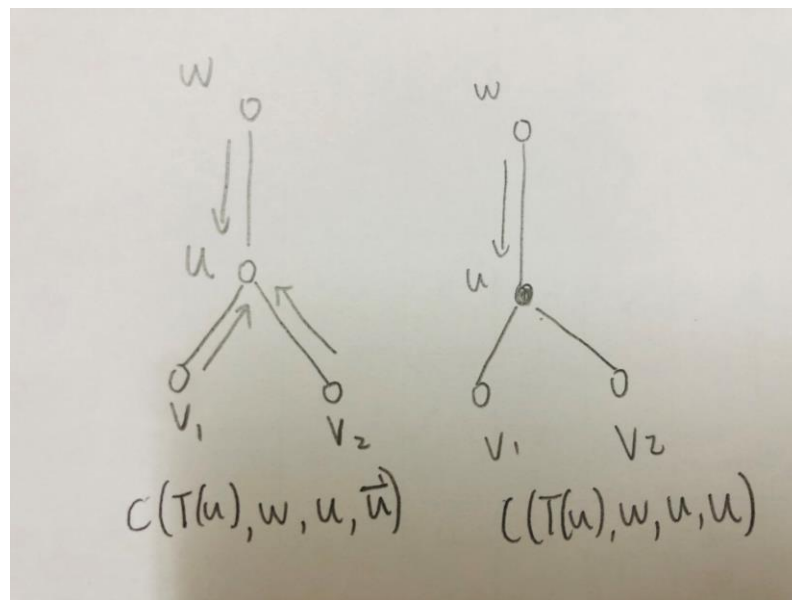
$$C(T(u), w, u, u) = wt(w) + wt(u) + \sum_{i=1}^m C(T(v_i))$$

$$C(T(u), u, w, \bar{u}) = wt(u) + \sum_{i=1}^m C(T(v_i), u, v_i)$$

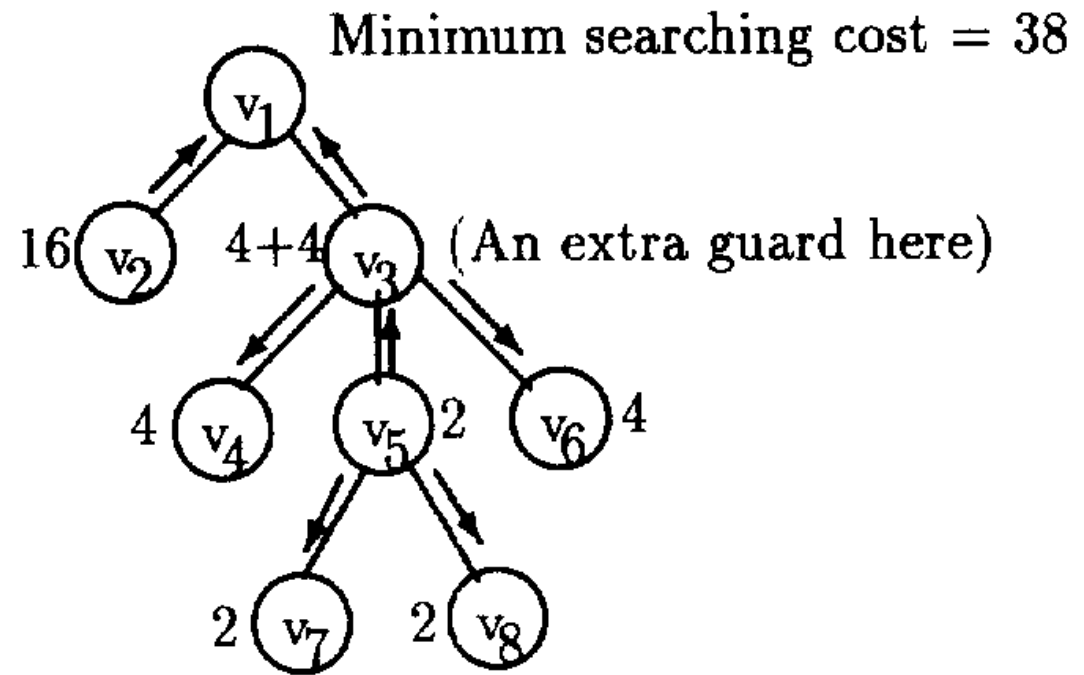
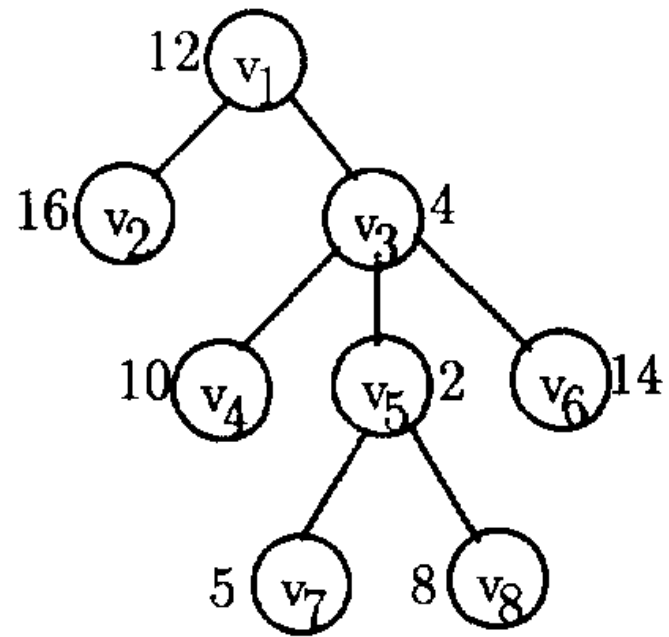
$$C(T(u), u, w, u) = 2wt(u) + \sum_{i=1}^m C(T(v_i))$$

$$C(T(u), w, u) = \min\{C(T(u), w, u, \bar{u}), C(T(u), w, u, u)\}$$

$$C(T(u), u, w) = \min\{C(T(u), u, w, \bar{u}), C(T(u), u, w, u)\}$$

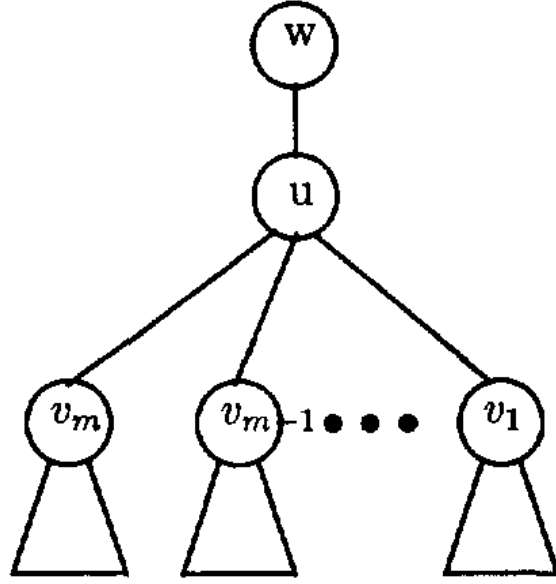


Example

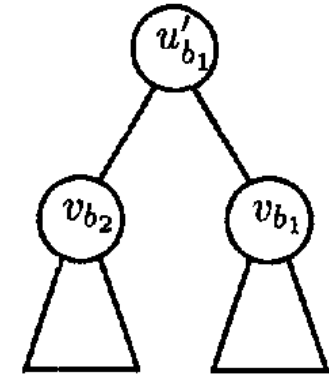


The Reduction of a General Tree to a Binary Tree

The parallel tree contraction technique assumes that the trees are binary trees. Therefore, we should transform a general tree to a binary tree. In this Section, we will describe a method that transforms a general tree to a binary tree for this single step searching problem by repeatedly applying the following procedure:



1. If node u has only one child, we add a dummy child node to node u .
2. If node u has two children nodes, do nothing.
3. If node u has m children nodes, $m > 2$, we add $m - 2$ contiguous pseudo nodes, $u'_{b_1}, u'_{b_2}, \dots, u'_{b_{m-2}}$, and $m - 1$ pseudo edges, $\overline{u_b u'_{b_{m-2}}}, \overline{u'_{b_{m-2}} u'_{b_{m-3}}}, \dots, \overline{u'_{b_2} u'_{b_1}}$, as in Figure 3, and the weighted assignment are $wt(u_b) = wt(u)$ and $wt(u'_{b_i}) = wt(u)$ for each $i (1 \leq i \leq m - 2)$. For convenience, we rename node v_i to v_{b_i} for $1 \leq i \leq m$. The subtree of v_{b_i} is the same as that of v_i , $i = 1, 2, \dots, m$.

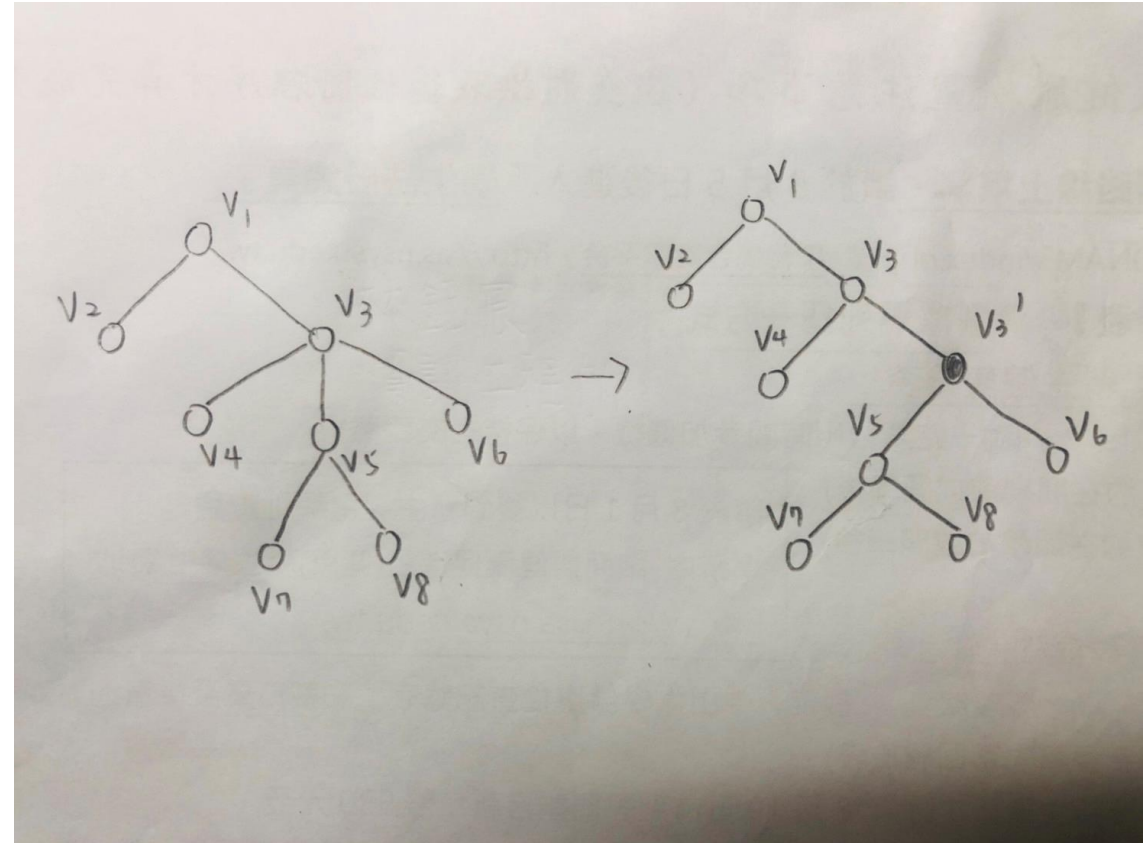


Real Node Rule: The cost evaluation rules for real nodes in $B(T(u_b), u)$ remain the same as those for the general tree $T(u)$.

Next, let us consider pseudo nodes. We stipulate that no extra guards can occupy any pseudo node.

Pseudo Node Rule: Let node u' be a pseudo node. Let x and y be the children nodes of u' . Let $p(u')$ be the parent node of u' . We compute the searching cost of a pseudo node as follows.

$$\begin{aligned} C(T(u'), u', p(u')) &= C(T(x), x, u') + C(T(y), y, u') \\ C(T(u'), p(u'), u') &= C(T(x), u', x) + C(T(y), u', y) \\ C(T(u')) &= C(T(x)) + C(T(y)) \end{aligned}$$

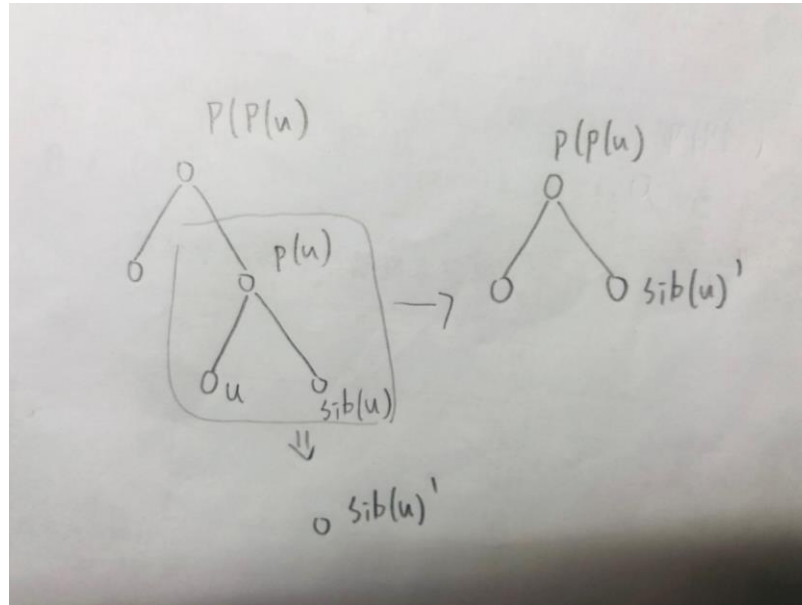


The Optimal Parallel Algorithm for the Weighted Single Step Searching Problem

First select a leaf node U , evaluate the searching costs of node U .

Next, partially evaluate the searching costs of parent node $p(U)$.

Then, we have to contract U , $p(U)$ and $\text{sib}(U)$ into one node and connect $\text{sib}(U)$ to $p(p(U))$.



Algorithm PWSSS

Step 1: Transform general tree $T(r)$ to binary tree $B(r_b)$.

Step 2: Number leaves in order from left to right except the first and the last nodes using Euler tour technique.

Step 3: Tree contraction operation:

While (numbered leaf nodes exist) **do**

Step 3.1: Concurrent process all odd-numbering leaf nodes which are left child nodes.

For each selected node, compute searching costs of those nodes as follows:

(a) Compute the searching costs of leaf node u by Rule 4.

(b) Compute the searching cost of node $p(u)$.

If $p(u)$ is a real node, applying the **Real Node Rule**.

If $p(u)$ is a pseudo node, applying the **Pseudo Node Rule**.

(c) Store partial searching cost information of nodes u and $p(u)$ into node $sib(u)$.

(d) Connect $sib(u)$ to $p(p(u))$.

Step 3.2: Concurrent process all the other odd-numbering leaves.

Compute the node searching cost same as Step 3.1.

Step 3.3: Renumber the remaining leaves in order from left to right.

Step 4: Compute the minimum searching cost of $BT(r)$.

(a) Compute the leftmost and rightmost leaf nodes as Step 3.1(a).

(b) Compute the minimum seahing cost of $BT(r)$ by Rule 1 and Rule 2.

Step 5: Backtrack the tree contraction.

Determine the searching direction by checking the searching cost of each node.

Step 6: Map the searching plan of $BT(r)$ to $T(r)$.

Each step of algorithm PWSSS runs in time $O(\log n)$ with $O(n)$ operations on EREW PRAM model. By Brent's theorem, we can show that the algorithm runs in time $O(\log n)$ with $n/\log n$ processors.

Brent's theorem:

For a given parallel algorithm A, $W(n)$ is total workload during time $T(n)$.

Then A can be run on p processors in time $t(n)=O(W(n)/p+T(n))$.

Conclusion

- The method is a systematic way to solve the problems that the sequential algorithm is based on dynamic programming and runs in linear time.
- This parallel algorithm for the problem runs in time $O(\log n)$ using $O(n/\log n)$ processors on EREW-PRAM. Thus, the parallel algorithms all are optimal speedup algorithms.